Implicitly learning token semantics

Gary Nan Tie, Mar 3, 2026

Abstract

A fast implicit recurrent deep learning model of semantic attention is proposed for the purposes of large language model next-token and sequence-to-sequence learning.

Introduction

The implicitRNN model [1] functions similarly to a vanilla RNN, processing sequence inputs one by one. For each time step i, where ti $\in$ R^p represents the i-th element in a sequence (t1, t2,$\cdots$, tm). The model input ui for implicitRNN is a concatenation of ti and the previous hidden state hi−1, akin to a vanilla RNN. The implicit prediction rule for implicitRNN is as follows:

h0 = 0; x0 =0;

ui = [ti hi−1]

x =Φ(Ax+Bui) (equilibrium equation)

ˆ yi(ui) = Cx+Dui (prediction equation)

hi = ˆyi(ui).

In this setup, the recurrent layer is replaced with an implicit structure comprising the equilibrium and prediction equations, where we learn matrices A, B, C, D and fixed points x.

Let score be a desired similarity function, like a kernel for example. Input ui corresponding to token ti, generates a unique fixed point si via the equilibrium equation. The hidden states h, get updated via the prediction equation. Akin to transformer attention looking at query-key similarity, fix i, normalize score(hi, sj) into probabilities, for hidden state hi and fixed points sj, to create a probability distribution over tokens, and output the most likely token. LLMs typically minimize perplexity. The wrinkle here is we can also regularize with the average Renyi α-entropy of these distributions. This enables robust next-token learning. Moreover, if we think of the latent tokens sj as possible meanings of the hidden state hi, then lowering average Renyi α-entropy sharpens and aligns semantics. Similar considerations (for score(hi, h'j), state hi corresponding to target token ti and state h'j corresponding to source token t'j) apply to sequence-to-sequence translation.

Note that perplexity attends to syntactic fidelity, while average Renyi α-entropy attends to semantic fidelity. The objective of a token learning problem can be tailored to any desired emphasis between the two.

Implicitly learning token semantics

Input: $\begin{cases} \text{target tokens } t_1, t_2, \ldots, t_m \\ \text{source tokens } t'_1, t'_2, \ldots, t'_n \end{cases}$

State Machine

initial state $h_0 = 0$

next state $h_i = f_s(h_{i-1}, t_i)$

output $o_{i+1} = f_o(h_i)$, $i \in [m]$

Implicit RNN

$h_0 = 0$, $s_0 = 0$

$$u_i = \begin{pmatrix} t_i \\ h_{i-1} \end{pmatrix}$$

$$s_i = \varphi(A s_i + B u_i)$$

$$\hat{y}_i(u_i) = C s_i + D u_i$$

$$h_i = \hat{y}_i(u_i) = f_s(h_{i-1}, t_i)$$

Given input token t_i, define output

$$o_{i+1} = t_{j_i^\beta} =: f_o(h_i)$$

where $\beta_{ij} = \underset{j \in [i]}{\text{softmax}}\ \text{score}(h_i, s_j)$

and $j_i^\beta = \underset{j \in [i]}{\arg\max}\ \beta_{ij}$,

score a desired similarity function e.g. a kernel.

Define average Renyi α-entropy of $\{t_i\}_{i=1}^m$

$$E(\{t_i\}) = \frac{1}{m} \sum_{i=1}^{m} H_\alpha^i$$

where $H_\alpha^i = H_\alpha(\{\beta_{ij}\}_{j=1}^i)$, H_α Renyi α-entropy.

(Similarly for $\{t'_j\}_{j=1}^n$, define β'_{ij}.)

Next-token learning problem

Let $\lambda > 0$, learn matrices A, B, C, D

and fixed points s_i to minimize

$$\{ \text{Perplexity}(o_2, \ldots, o_{m+1}) + \lambda E(\{t_i\}) \}$$

Next-token prediction

Initialization: let t_1 = the empty token
or a seed token

Given $[t_1]$, define $t_2 = o_2$

Given $[t_1, t_2]$, define $t_3 = o_3$

Given $[t_1, t_2, t_3]$, define $t_4 = o_4$

and so on, autoregressively generating the next token.

Sequence to sequence learning

For $\{t_i\}$, learn A, B, C, D and s_i to minimize

$$\{ \text{Perplexity}(o_2, \ldots, o_{m+1}) + \lambda E(\{t_i\}) \}$$

Similarly for $\{t'_j\}$, learn A', B', C', D' and s'_j to minimize

$$\{ \text{Perplexity}(o'_2, \ldots, o'_{n+1}) + \lambda' E(\{t'_j\}) \}$$

Consequently we know $\{h_i\}_{i=1}^m$ and $\{h'_j\}_{j=1}^n$

Define $\gamma_{ij} = \underset{j \in [n]}{\text{softmax}}\ \text{score}(h_i, h'_j)$, $i \in [m]$

and $j_i^\gamma = \underset{j \in [n]}{\arg\max}\ \gamma_{ij}$

Given input t_i, $i \in [m]$, define output $o_i = t'_{j_i^\gamma}$

So $t_1, \ldots, t_m$ is translated to $o_1, \ldots, o_m$, $o_i \in \{t'_j\}_{j=1}^n$

References

[1] 'The Extrapolation Power of Implicit Models'
Juliette Decugis, Alicia Y. Tsai,
Max Emerling, Ashwin Ganesh, Laurent El Ghaoui
arXiv:2407.14430v1 [cs.LG] 19 Jul 2024